

```
manchodu@Srini:~$ cd OSLAB
manchodu@Srini:~/OSLAB$ nano memory_Layout.c
manchodu@Srini:~/OSLAB$ gcc -o memory_layout memory_layout.c
cc1: fatal error: memory_layout.c: No such file or directory
compilation terminated.
manchodu@Srini:~/OSLAB$ gcc -o memory_Layout memory_layout.c
cc1: fatal error: memory_layout.c: No such file or directory
compilation terminated.
manchodu@Srini:~/OSLAB$ gcc -o memory_Layout memory_Layout.c
manchodu@Srini:~/OSLAB$ ./memory_Layout
=====
LINUX PROCESS MEMORY LAYOUT
=====

Process ID (PID): 537

CODE SEGMENT
Address of code_function : 0x5c2d13ab61c9

GLOBAL VARIABLES
Address of global_var      : 0x5c2d13ab9010
Address of global_uninitialized : 0x5c2d13ab9020

STATIC VARIABLES
Address of static_var      : 0x5c2d13ab9014
Address of static_local_var : 0x5c2d13ab9018

HEAP
Address of heap_var : 0x5c2d437712a0

STACK
Address of stack_var : 0x7fff6624218c

=====
Keep this program running to inspect /proc.
PID = 537
=====
```

```

manchodu@Srini:~/OSLAB$ cat memory_Layout.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Global variable - initialized */
int global_var = 100;

/* Global variable - uninitialized */
int global_uninitialized;

/* Static variable */
static int static_var = 200;

/* Function - stored in Code/Text segment */
void code_function()
{
    printf("Inside code_function()\n");
}

int main()
{
    /* Local variable - Stack */
    int stack_var = 300;

    /* Static local variable - Data segment */
    static int static_local_var = 400;

    /* Heap variable */
    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 500;

    printf("=====\n");
    printf("      LINUX PROCESS MEMORY LAYOUT\n");
    printf("=====\n\n");

    printf("Process ID (PID): %d\n\n", getpid());

    printf("CODE SEGMENT\n");
    printf("Address of code_function : %p\n",
        (void *)code_function);

    printf("\nGLOBAL VARIABLES\n");
    printf("Address of global_var      : %p\n",
        (void *)&global_var);

    printf("Address of global_uninitialized : %p\n",
        (void *)&global_uninitialized);

    printf("\nSTATIC VARIABLES\n");

```

```

printf("=====\n");
printf("      LINUX PROCESS MEMORY LAYOUT\n");
printf("=====\n\n");

printf("Process ID (PID): %d\n\n", getpid());

printf("CODE SEGMENT\n");
printf("Address of code_function : %p\n",
      (void *)code_function);

printf("\nGLOBAL VARIABLES\n");
printf("Address of global_var      : %p\n",
      (void *)&global_var);

printf("Address of global_uninitialized : %p\n",
      (void *)&global_uninitialized);

printf("\nSTATIC VARIABLES\n");
printf("Address of static_var      : %p\n",
      (void *)&static_var);

printf("Address of static_local_var : %p\n",
      (void *)&static_local_var);

printf("\nHEAP\n");
printf("Address of heap_var : %p\n",
      (void *)heap_var);

printf("\nSTACK\n");
printf("Address of stack_var : %p\n",
      (void *)&stack_var);

printf("\n=====\n");
printf("Keep this program running to inspect /proc.\n");
printf("PID = %d\n", getpid());
printf("=====\n");

/* Keep process alive */
while (1)
{
    sleep(10);
}

free(heap_var);

return 0;
}

```